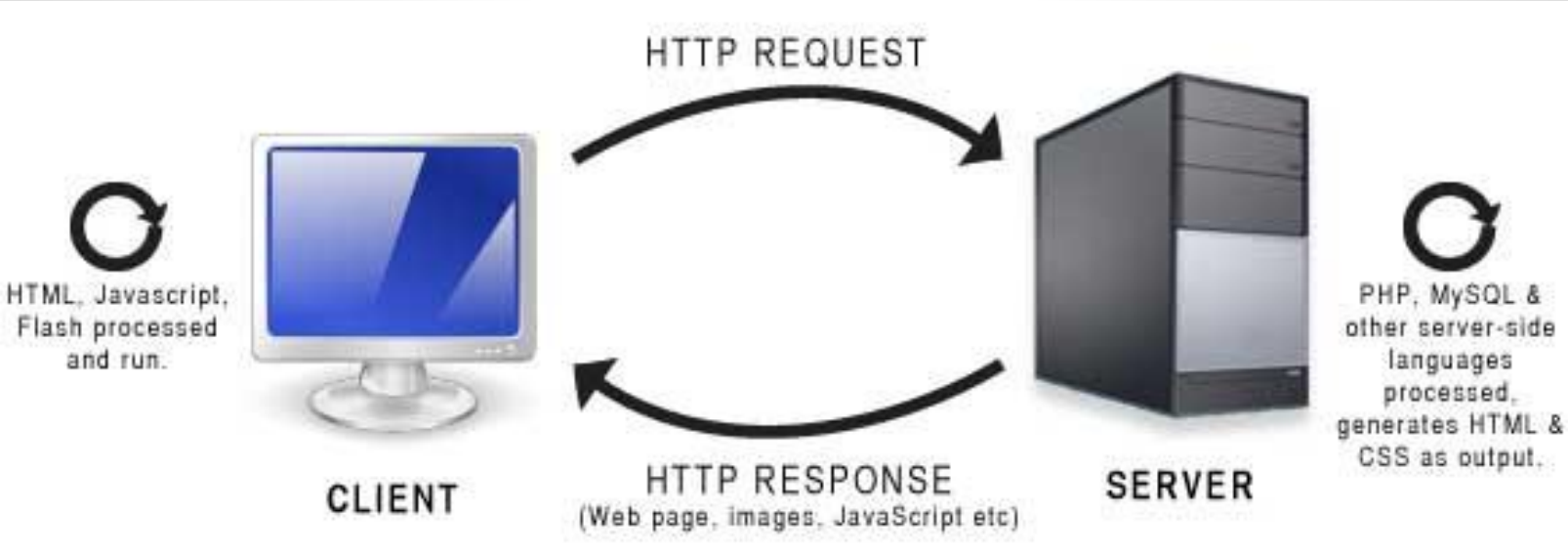


Plataforma de Experimentación A/B en Redes de Entrega de Contenido

Guillermo Pereyra Irujo

Directora: Dra. Mariana del Fresno
Codirector: Ing. Pablo Del Grosso

Arquitectura de la WWW



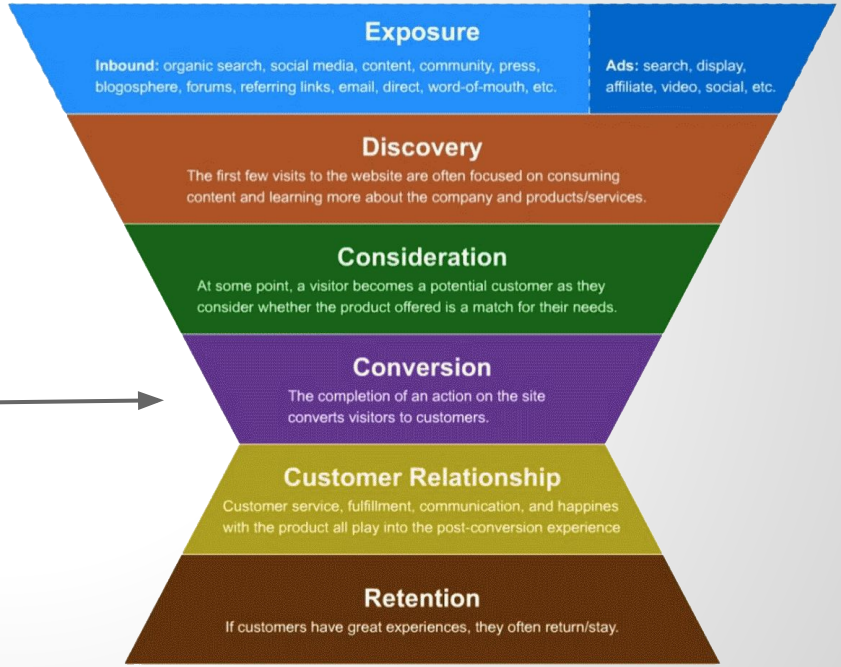
Embudo del marketing digital

Exposición

El usuario ha percibido una característica o funcionalidad del producto

Conversión

El usuario ha hecho una operación de interés para el negocio



Experimentación A/B



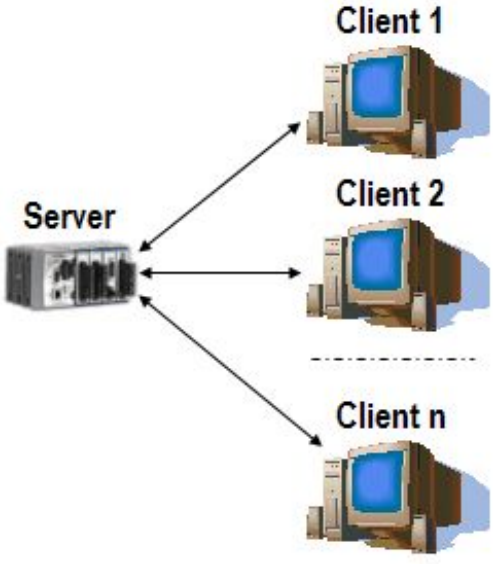
Esencia de la implementación



El dado se guarda para mostrar siempre la misma variación

Variación del contenido

En el servidor
Cada pedido llega al servidor y se entrega el contenido que corresponda



En el cliente
Se entrega siempre el mismo contenido y la variación se hace en el cliente

Variación en el servidor

Cliente envía pedido al servidor

Servidor tira dado si no hay

Servidor elige variación según dado

Servidor genera versión de respuesta

Cliente recibe respuesta y muestra

Tiempos de entrega del contenido

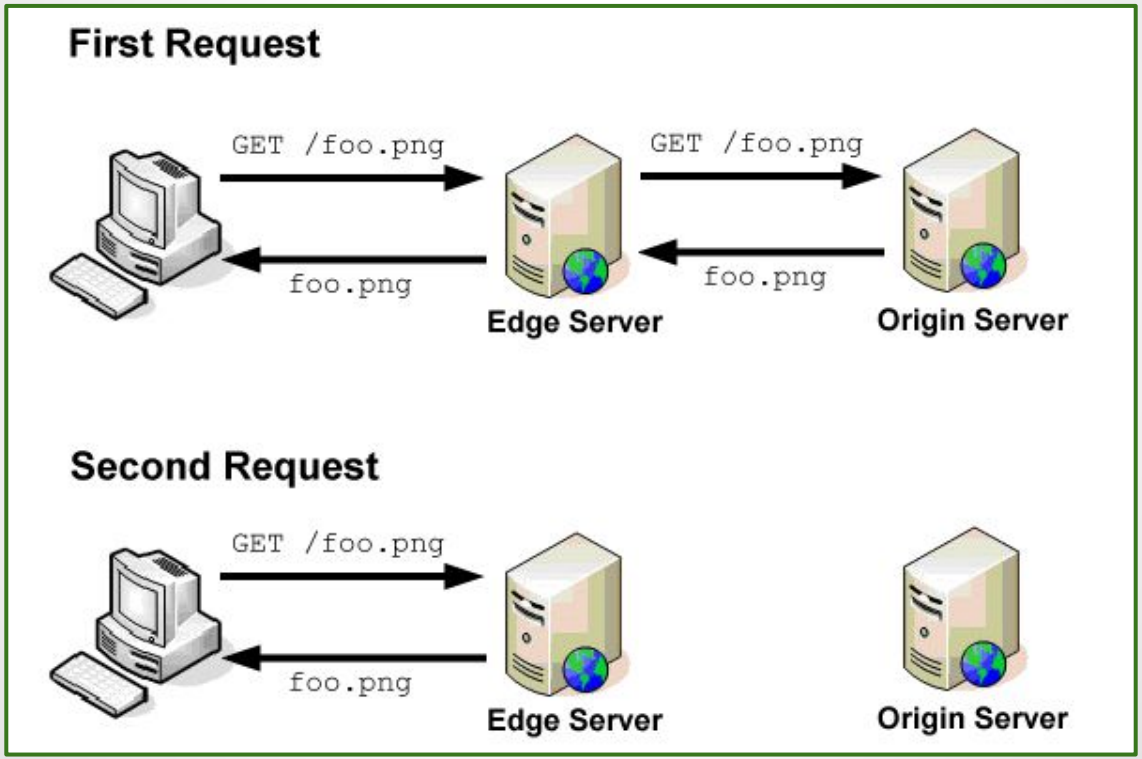
Tiempo de transferencia

$$\textit{Tamaño del objeto} \times \textit{Velocidad de transferencia} + \textit{Tiempo al primer byte}$$

Tiempo al primer byte

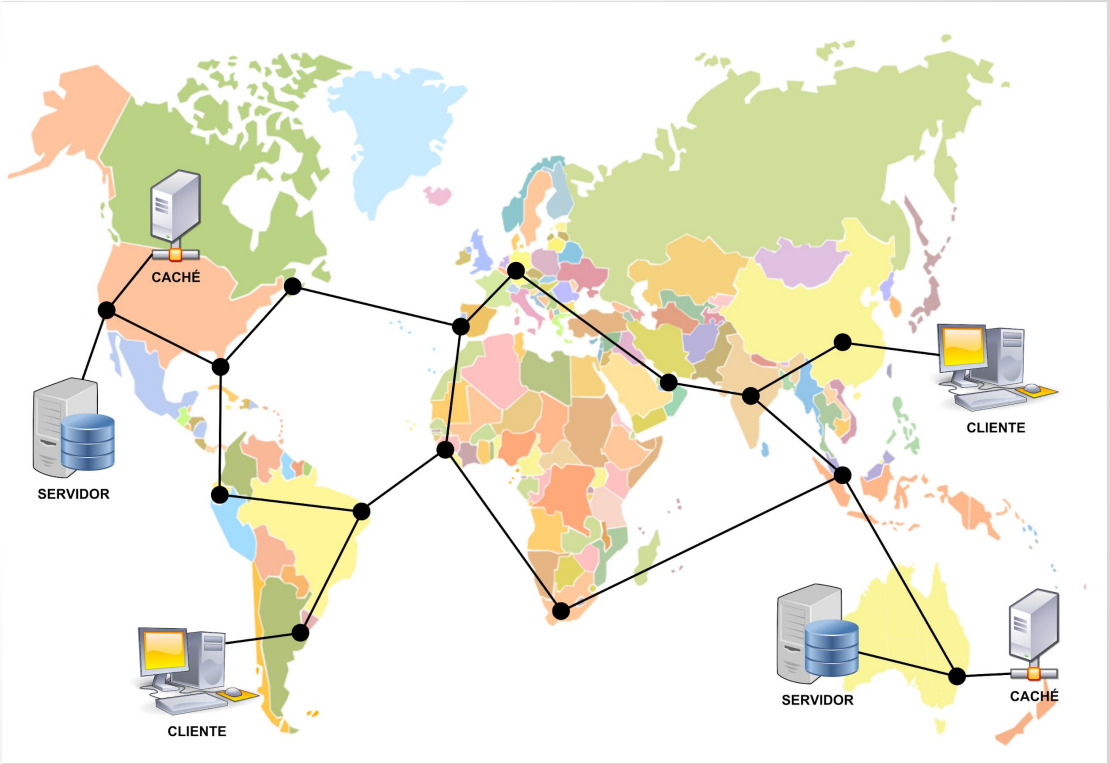
$$\textit{Latencia del pedido} + \textit{Tiempo de generación} + \textit{Latencia de la respuesta}$$

Caching de contenido



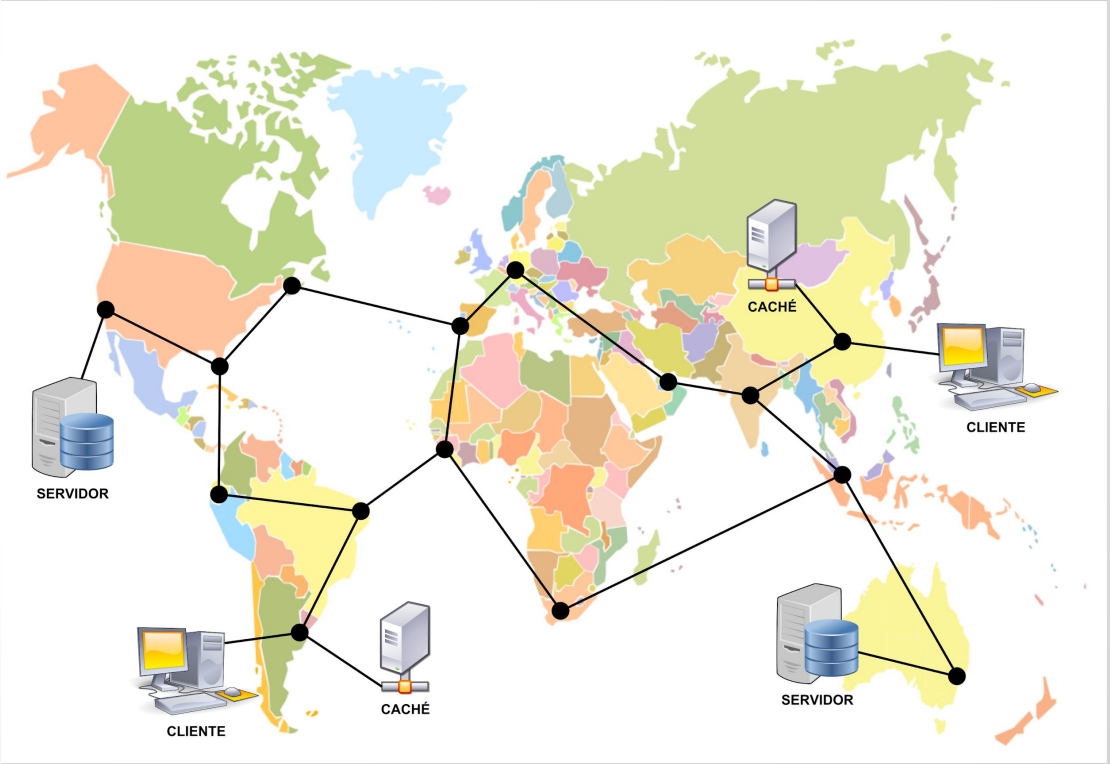
Caching del lado del servidor

Reduce el tiempo de generación



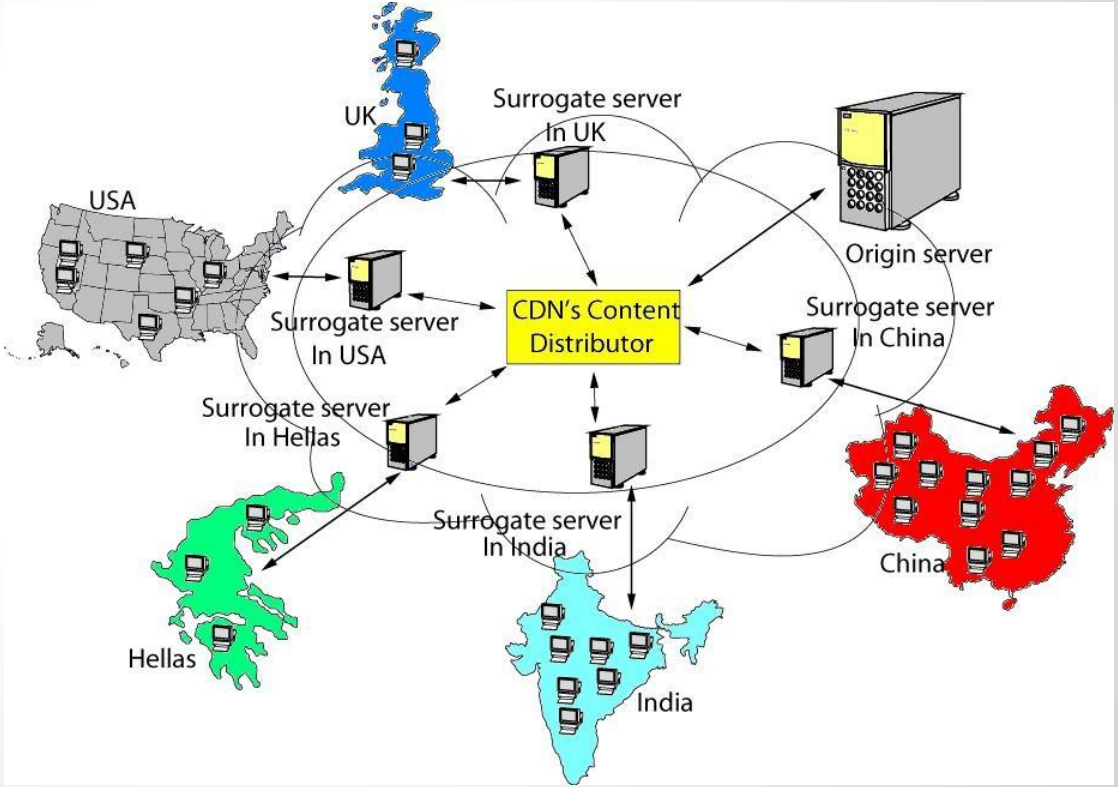
Caching del lado del cliente

Reduce la latencia



Red de entrega de contenido

Servidores de caching distribuidos globalmente



Variación en el servidor con CDN

Si se cachea en la CDN, no se puede elegir en el servidor la versión del documento

Si cada pedido llega hasta el servidor para elegir la versión del documento, no se usa la CDN

Variación en el cliente

Cliente envía pedido al servidor

Servidor entrega respuesta sin variación

Cliente recibe respuesta

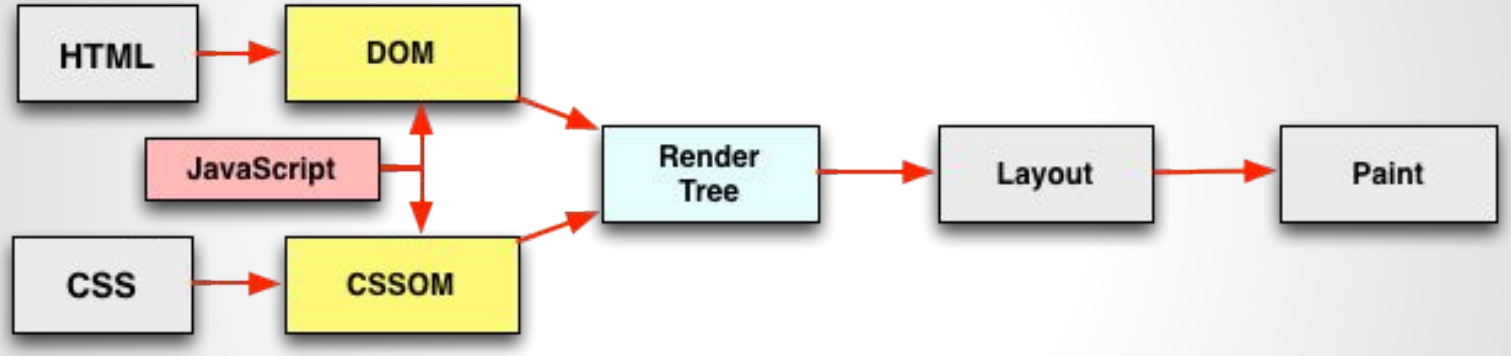
Cliente tira dado si no hay

Cliente elige variación según dado

Cliente aplica variación en respuesta si corresponde

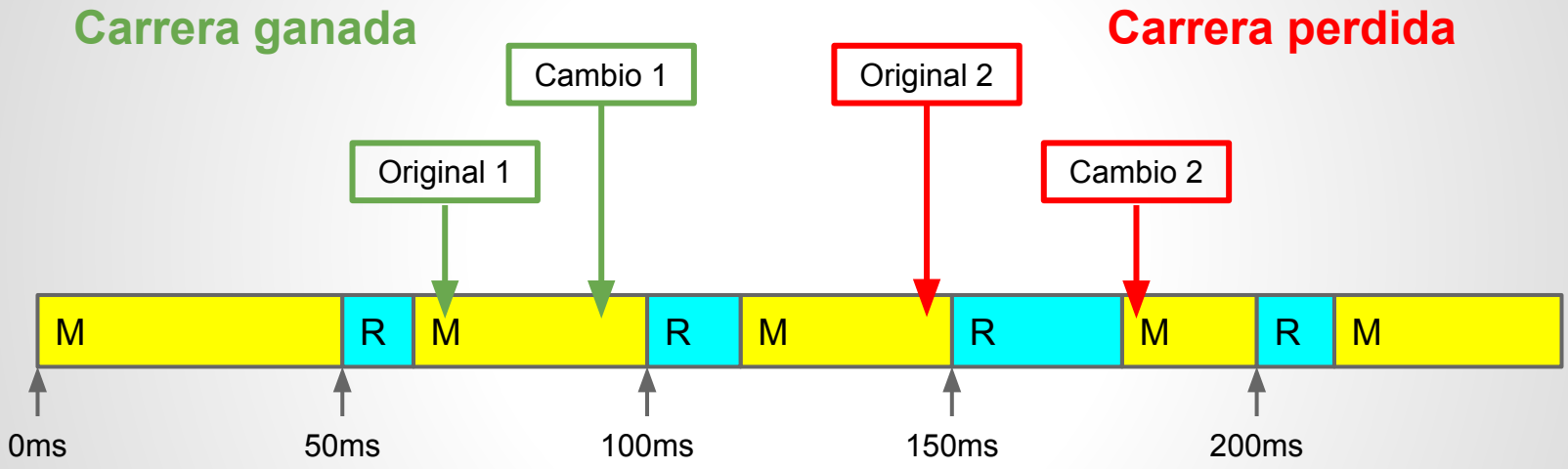
La respuesta del servidor es constante y cacheable!

Hilos del navegador



Etapa de modelado	Etapa de *rendering*

Condición de carrera



Si el elemento original y el cambio entran en el DOM **en el mismo período** de *rendering*, el cambio **no es percibido**

Si el elemento original y el cambio entran en el DOM **en distintos períodos** de *rendering*, el cambio **es percibido**

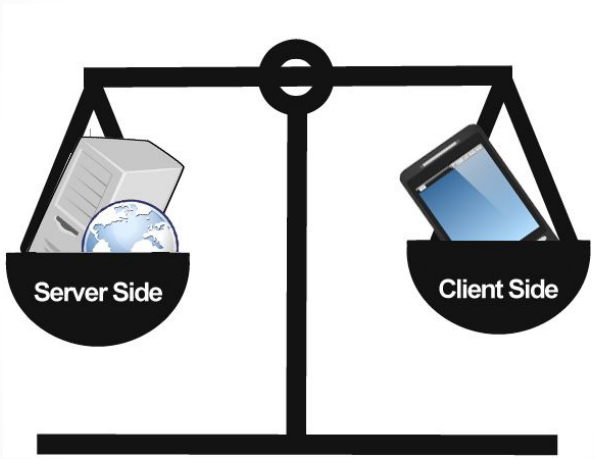
Doble implementación

Cada cambio es implementado dos veces, una en el cliente y una en el servidor

- en otro lenguaje portación de código
- por otra gente comunicación
- con otra forma imperativa vs. declarativa

Problema

No se puede variar en el servidor porque se inutiliza la CDN



No se quiere variar en el cliente porque afecta el producto y cuesta el doble

La CDN se puede guionar

```
<esi:choose>

  // cuando hay una sesión, saludar al usuario
  <esi:when test="$exists($(HTTP_COOKIE{'username'}))">
    <html>
      <body>
        Bienvenido $(HTTP_COOKIE{'username'})!
      </body>
    </html>
  </esi:when>

  // de lo contrario, ofrecer iniciar sesión
  <esi:otherwise>
    <html>
      <body>
        <a href="/login">Iniciar sesión</a>
      </body>
    </html>
  </esi:otherwise>

</esi:choose>
```

Edge Side Includes (ESI)
Lenguaje de etiquetas
para componer contenido
en servidores de caching

Solución



Hacer las variaciones de contenido en la CDN

El servidor no necesitará procesar cada pedido y sus respuestas serán cacheables

El cliente recibirá la variación que le corresponda y no deberá procesar nada

Componentes



Variación de página entera

```
/pagina
    <esi:choose>

      <esi:when test="$(dado) == 'A'">
        <esi:include src="/pagina?version=A"/>
      </esi:when>

      <esi:otherwise>
        <esi:include src="/pagina?version=B"/>
      </esi:otherwise>

    </esi:choose>

/pagina?version=A
    <html>
      <body>Contenido de la página versión A</body>
    </html>

/pagina?version=B
    <html>
      <body>Contenido de la página versión B</body>
    </html>
```

Implementación

La respuesta es un documento ESI que **incluye** una versión según el dado.

Las tres respuestas son cacheables.

Problema

Las partes comunes se generan 2 veces y se guardan 2 veces en caché.

La infraestructura es puesta en marcha 3 veces.

Variación de bloques

```
<html>
  <body>

    <esi:choose>
      <esi:when test="$(dado) == 'A'">

        Contenido de la página versión A

      </esi:when>
      <esi:otherwise>

        Contenido de la página versión B

      </esi:otherwise>
    </esi:choose>

  </body>
</html>
```

Implementación
La respuesta es un documento ESI que **selecciona** una versión según el dado.
La respuesta es constante y cacheable y nada se hace más de una vez.

Definición de experimentos

```
<experimentos>

    <experimento>
        <nombre>Color de botón publicar</nombre>
        <variaciones>
            <variacion nombre="verde" porcentaje="50"/>
            <variacion nombre="naranja" porcentaje="50"/>
        </variaciones>
    </experimento>

    <experimento>
        <nombre>Mapa en anuncio</nombre>
        <variaciones>
            <variacion nombre="con mapa" porcentaje="10"/>
            <variacion nombre="sin mapa" porcentaje="90"/>
        </variaciones>
    </experimento>

</experimentos>
```

Selección de variación al inicio

```
<esi:assign name="dado" value="$tirarDado"/>
<esi:assign name="experimento1" value="$variacion($(dado), 50)"/>
<esi:assign name="experimento2" value="$variacion($(dado), 25)"/>
<esi:assign name="experimento3" value="$variacion($(dado), 40)"/>

<html>
  <body>
    <h1>Página principal</h1>
    <a href="/publicar">Publique su
      <esi:choose>

        <esi:when test="$(experimento1) == 'TRATAMIENTO'">
          aviso clasificado
        </esi:when>

        <esi:otherwise>
          anuncio clasificado
        </esi:otherwise>

      </esi:choose>
    </a>
  </body>
</html>
```

Implementación

Al inicio del documento, se decide qué variación de cada experimento se aplica.

Problema

Se calcula la variación de experimentos para los que no hay una variación local.

Selección de variación por demanda

```
<esi:assign name="dado" value="$tirarDado"/>

<html>
  <body>
    <h1>Página principal</h1>
    <a href="/publicar">Publique su

      // selección de variación de testAvisoAnuncio
      <esi:assign name="testAvisoAnuncio" value="$variacion($(dado), 50)"/>

      // condicionamiento del contenido según testAvisoAnuncio
      <esi:choose>
        <esi:when test="$(testAvisoAnuncio) == 'TRATAMIENTO'">
          aviso clasificado
        </esi:when>
        <esi:otherwise>
          anuncio clasificado
        </esi:otherwise>
      </esi:choose>

    </a>
  </body>
</html>
```

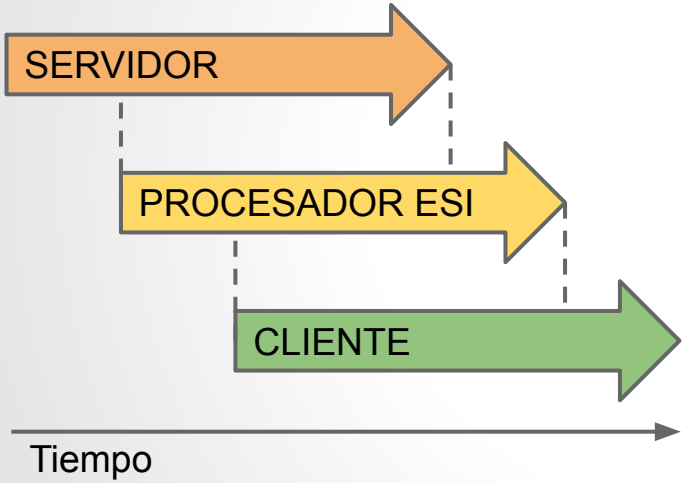
Implementación

Se evalúa el dado inmediatamente antes de la variación.

Problema

La evaluación puede quedar condicionada y faltar en una variación posterior.

Procesamiento imbricado



Cada componente emite salida antes de terminar de procesar su entrada

Definición del conjunto a priori

```
<experimentos>

    <experimento>
        <nombre>Color de botón publicar</nombre>
        <variaciones>
            <variacion nombre="verde" porcentaje="50"/>
            <variacion nombre="naranja" porcentaje="50"/>
        </variaciones>
        <paginas>
            <pagina nombre="/"/>
            <pagina nombre="/listado"/>
        </paginas>
    </experimento>

    <experimento>
        <nombre>Mapa en anuncio</nombre>
        <variaciones>
            <variacion nombre="con mapa" porcentaje="10"/>
            <variacion nombre="sin mapa" porcentaje="90"/>
        </variaciones>
        <paginas>
            <pagina nombre="/anuncio"/>
        </paginas>
    </experimento>

</experimentos>
```

Implementación

Se indica *a priori* la cobertura del experimento en la definición.

Problema

La definición y la implementación quedan acopladas y la definición *a priori* no siempre es posible.

El conjunto se conoce después de evaluar el cuerpo del documento, pero debe ser emitido antes.

Distribución de recursos externos

```
<html>
  <head>

    // Hojas de estilo
    <link rel="stylesheet" href="site-stylesheet.css"/>
    <link rel="stylesheet" href="page-stylesheet.css"/>

  </head>
  <body>

    // Contenido visible de la página
    <h1 id="titulo">Avisos Clasificados</h1>
    <div class="seccion">
      <p>Lorem ipsum dolor sit amet, consectetur adipiscing</p>
      <ul>
        <li>eiusmod tempor incididunt ut labore</li>
        <li>Ut enim ad minim veniam</li>
      </ul>
      <p><img src="/imagenes/logo.png"/>
    </div>

    // Guiones
    <script src="google-analytics.js"></script>
    <script src="jquery.js"></script>

  </body>
</html>
```

Hojas de estilo antes
Se cargan asincrónicamente y
son necesarias para el rendering
→ se cargan lo más temprano posible

Guiones después
Se cargan sincrónicamente y
no son necesarios para el rendering
→ se cargan lo más tarde posible

Posición del código de registro

```
<html>
  <body>

    // Contenido visible del cuerpo
    <h1 id="titulo">Avisos Clasificados</h1>
    <p>Lorem ipsum dolor sit amet, consectetur adipiscing</p>

    // exportación a javascript del resultado de la selección
    <script>
      tests['test1'] = '<esi:vars name="test1"/>';
    </script>

    // rutina de tracking
    <script src="googleanalytics.js"></script>
    <script>
      googleAnalytics.addDimensions(tests);
      googleAnalytics.trackPageView();
    </script>

    // otros scripts
    <script src="jquery.js"></script>
    <script src="lightbox.js"></script>

  </body>
</html>
```

Problema

El código de registro en el cliente debe conocer la variación elegida y está en una ubicación arbitraria del documento.

El conjunto se conoce después de evaluar el cuerpo del documento, pero debe ser emitido antes.

Inversión de control

Se definen las acciones específicas ante dos eventos evaluación de experimentos y exportación de datos

Se entrega el control a una rutina ya definida que controla cuándo se ejecutan.

Se puede emitir el código en el orden opuesto al de la ejecución.

```
// contenido emitido antes y ejecutado después
function main(fn_evaluar, fn_exportar) {

   fn_evaluar()

   <html>
      <body>

         <esi:when test="experimento1 = A">
            <contenido de la versión A>
         </esi:when>

         fn_exportar()

         <script src="jquery.js"/>
         <script src="lightbox.js"/>

      </body>
   </html>

}

// contenido emitido después y ejecutado antes
main(
   function() {
      dado = tirar_dado()
      versiones = seleccionar_versiones(dado)
   },
   function() {
      print to_javascript(versiones)
   }
)
```

Consistencia de navegación

Implementación

En cada cliente se almacena una lista de pares Experimento = Variación.

Problema

La lista crece indefinidamente.

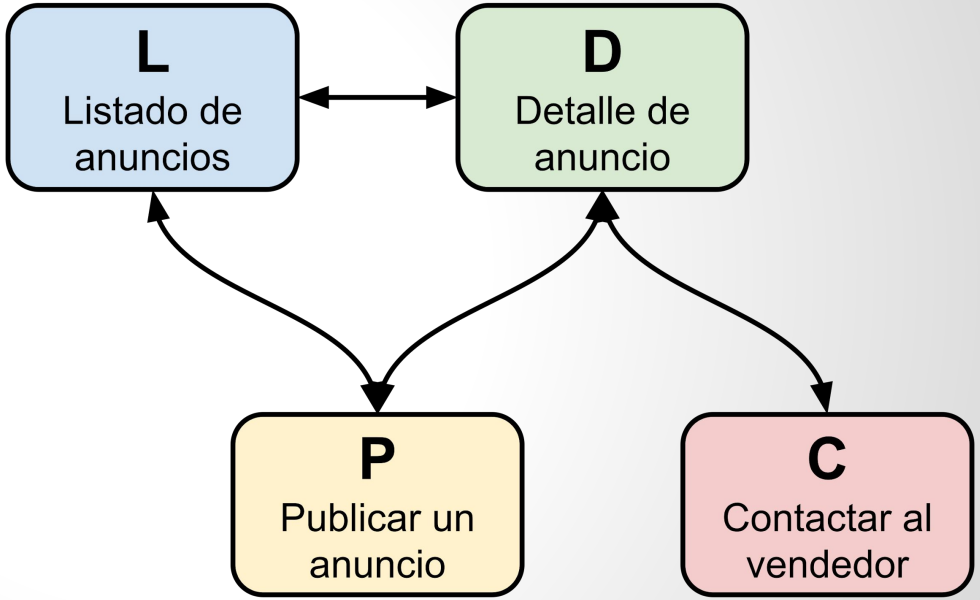
No se puede hacer garbage collection porque no se conoce *a priori* el conjunto de experimentos.

Registro de exposición y conversión

Cada visita a una página se registra.

Cada acción del usuario se registra.

Cada registro puede contener etiquetas asociadas.



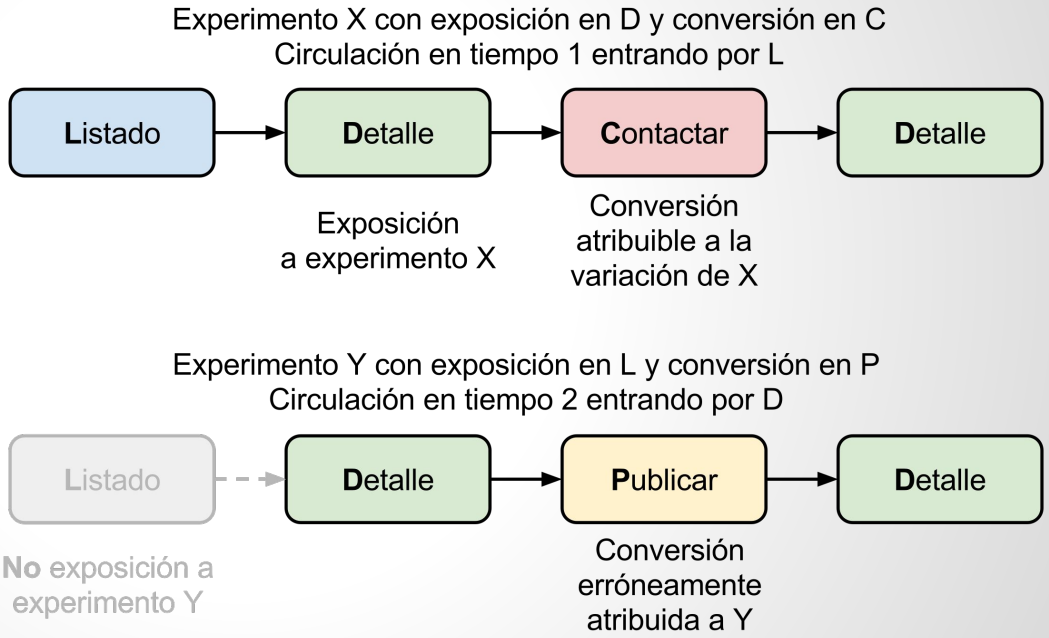
Dado único y constante

Implementación

Se tira un solo dado, se etiquetan los registros con el dado, y se asume exposición.

Problema

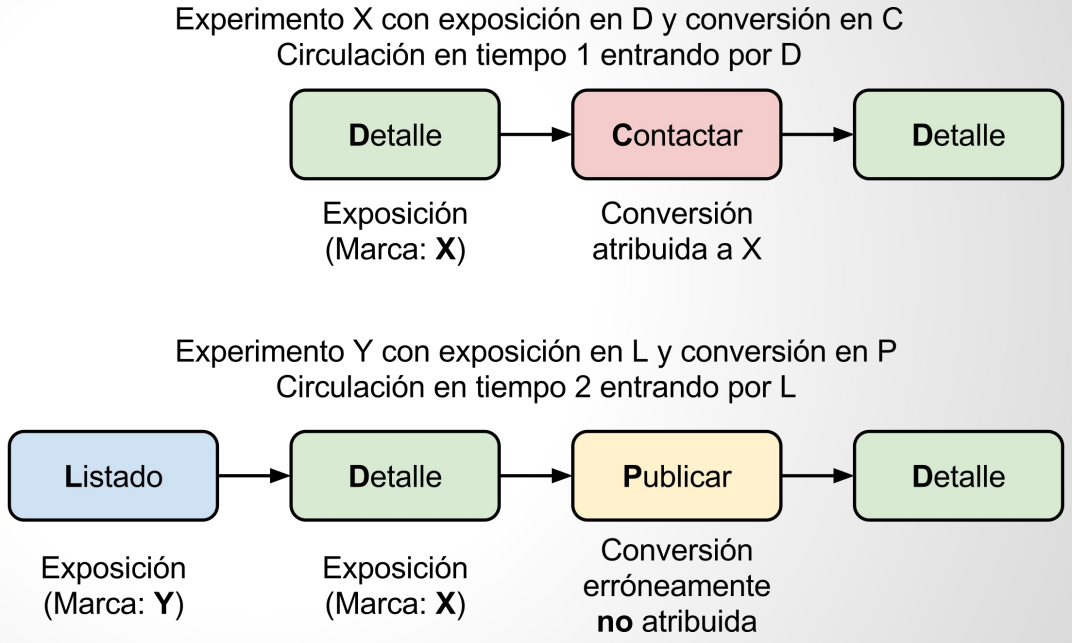
Si la conversión ocurre en una página distinta, es incorrectamente atribuida a la exposición.



Marca de exposición

Implementación
Se agrega una marca indicando la exposición.

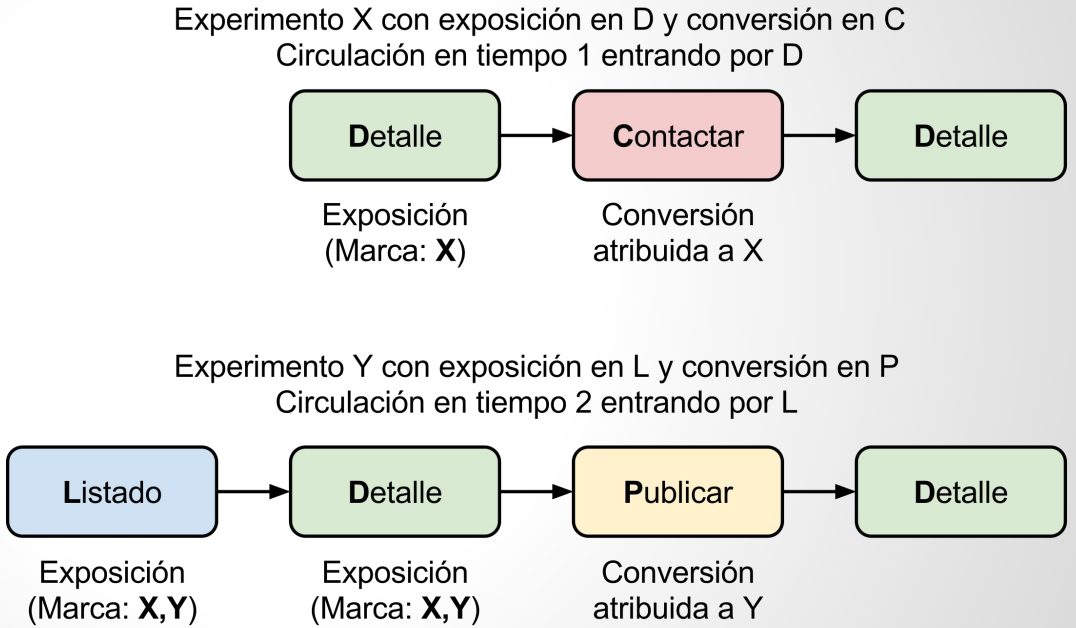
Problema
Si hay dos experimentos corriendo, la marca de uno puede pisar la marca del otro.



Lista de marcas de exposición

Implementación
Se convierte la marca en una lista de marcas de exposición.

Problema
La lista crece indefinidamente y tiene el mismo problema que usar un solo dado evita.

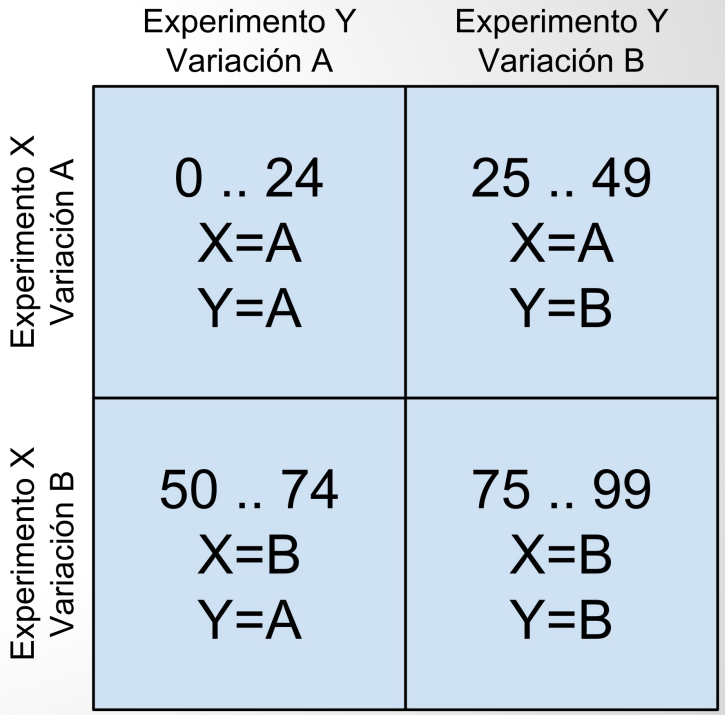


Distribución de probabilidad

Problema

El reparto manual del espacio de probabilidad entre las variaciones impone una dificultad al operador y un riesgo a los experimentos.

Hay que dispersar automáticamente las variaciones en cada experimento.



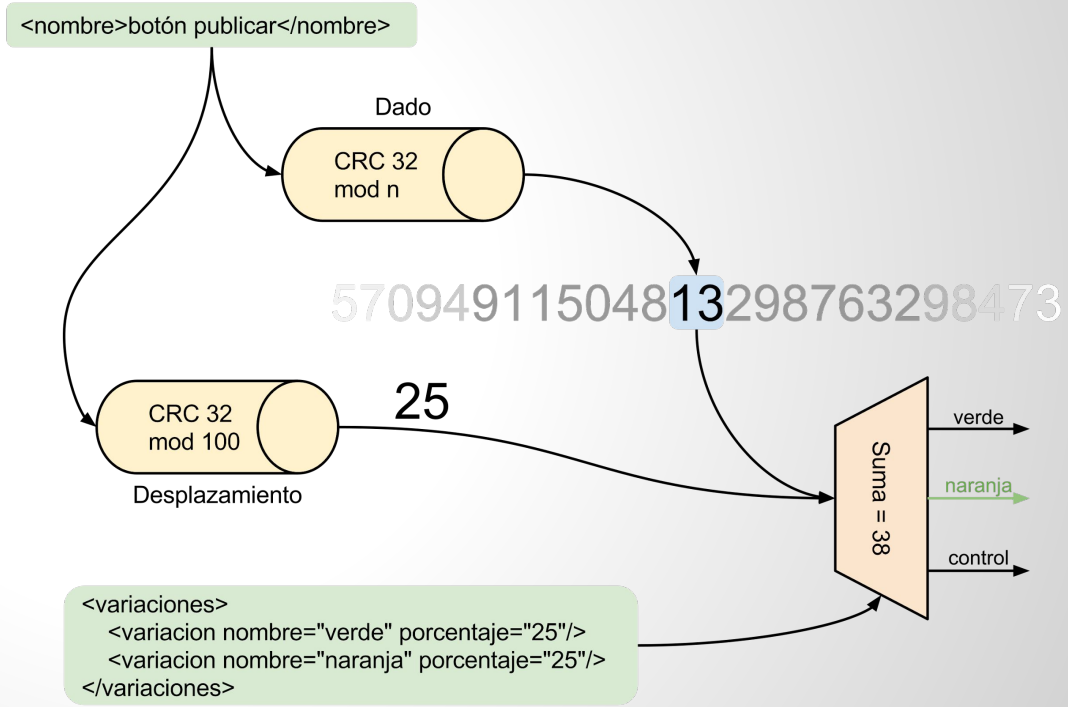
Interpretación dinámica del dado

Solución

Tirar un dado único y constante e interpretarlo de una manera nueva con cada experimento.

Implementación

Un dado de *n* dimensiones, de las cuales se elige una para cada experimento, bajando el acomplamiento a *1/n*. Una rotación adicional lo baja aún más.



Resultados

- ✓ El problema fue resuelto
- ✓ El registrador externo permitió ahorrar desarrollo
- ✓ La interfaz de desarrollo resultó no invasiva
- ✓ Variar el contenido por bloques resultó más eficiente
- ✓ La inversión de control resuelve las limitaciones de la emisión sincrónica en escenarios de procesamiento imbricado
- ✓ El dado constante permite quitar la acción del componente menos controlado
- ✓ La CDN fue incorporada a la arquitectura

Conclusiones

Las opciones tradicionales de Experimentación A/B tienen un costo en calidad de servicio o de producto

Contribución inmediata
Resolución del problema, actualmente en uso

Contribución mediata
Incorporación de la CDN a la arquitectura elevándola un orden de magnitud

Trabajo futuro

- Nuevos medios de registro, incluso en la misma CDN
- Distribución dinámica de las probabilidades
- Nuevas aplicaciones para la CDN
  - Restricción de acceso al contenido
  - Distribución de tráfico en sitios paralelos



¡Gracias!